Name : Shivraj Vaibhav Jagtap
Roll No : EN24107060
Class : SY - AI \& DS
Subject : Python for Visual AI

Assignment 10:

Deploy a crop recommendation ML model and automate predictions, and perform the tasks like Train ML model, save model using joblib, create inference script, and automating predictions from new input files.

Step 1 : Linear Regression Model for House Price Prediction

We build a **Linear Regression model** to predict house prices using:

- Size (sq. ft.)
- Number of bedrooms

Dataset Representation

$$X = \begin{bmatrix} 1000 & 2 \\ 1500 & 3 \\ 2000 & 3 \\ 2500 & 4 \\ 3000 & 4 \end{bmatrix}$$

$$y = \begin{bmatrix} 200000 \\ 300000 \\ 350000 \\ 450000 \\ 500000 \end{bmatrix}$$

Model Equation

Simple Linear Regression (Single Feature)

$$y = mx + c$$

Where:

- m = slope
- c = intercept

Slope:

$$m = \frac{n\sum xy - (\sum x)(\sum y)}{n\sum x^2 - (\sum x)^2}$$

Intercept:

$$c = \frac{\sum y - m\sum x}{n}$$

Prediction:

$$\hat{y} = mx + c$$

Multiple Linear Regression

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2$$

Matrix form:

$$\hat{y} = X\beta$$

Optimal coefficients:

$$\beta = (X^T X)^{-1} X^T y$$

In [1]:
```
import joblib
from sklearn.linear_model import LinearRegression

# Simple data: [size, bedrooms] -> price
X = [[1000, 2], [1500, 3], [2000, 3], [2500, 4], [3000, 4]]  # features
y = [200000, 300000, 350000, 450000, 500000]  # prices

# Train model
model = LinearRegression()
model.fit(X, y)

# Save model
joblib.dump(model, 'model.pkl')
print("Model saved!")
```

```
Model saved!
```

Step 2 : Flask API for House Price Prediction

We deploy a trained **Linear Regression model** using a **Flask API** to predict house prices based on:

- Size (sq. ft.)
- Number of bedrooms

System Architecture

The workflow of the system is:

$$\text{Client Request} \rightarrow \text{Flask API} \rightarrow \text{ML Model} \rightarrow \text{Response}$$

Model Loading

The trained model is loaded using `joblib`:

$$\text{model} = \text{joblib.load('model.pkl')}$$

- Loads pre-trained model parameters
- Avoids retraining every time

API Definition

We define a Flask application:

$$\text{app} = \text{Flask}(__name__)$$

- Initializes the web server

Endpoint Creation

We create a POST endpoint:

$$\text{predict}$$

$$\text{Method} = \text{POST}$$

- Accepts JSON input
- Returns prediction as JSON

Input Representation

The API expects input in JSON format:

$$\text{Input} = \begin{bmatrix} \text{size} \\ \text{bedrooms} \end{bmatrix}$$

Example:

```
{
  "size": 2000,
  "bedrooms": 3
}
```

Prediction Function

The prediction is computed as:

$$\hat{y} = f(\text{size}, \text{bedrooms})$$

Using model:

$$\hat{y} = \text{model.predict}([[x_1, x_2]])$$

Where:

- x_1 = size
- x_2 = bedrooms

Output Representation

The API returns:

$$\text{Output} = [\,\text{price}\,]$$

Example:

```
{
  "price": 350000.00
}
```

Server Execution

The server runs on:

$$\text{Host} = 127.0.0.1$$

$$\text{Port} = 2000$$

Access URL:

$$\text{http://127.0.0.1:2000}$$

Thread-Based Execution

To run Flask in background:

$$\text{Thread} \rightarrow \text{run_flask}()$$

- Prevents blocking main program
- Enables parallel execution

Response Function

The response is formatted as:

$$\text{Response} = \text{jsonify}(\{\text{price} : \hat{y}\})$$

With rounding:

$$\hat{y} = \text{round}(\hat{y}, 2)$$

In [2]:

```
from flask import Flask, request, jsonify
import joblib
import threading
import time

# Load model
model = joblib.load('model.pkl')

app = Flask(__name__)

@app.route('/predict', methods=['POST'])
def predict():

    data = request.get_json()
    size = data['size']
    bedrooms = data['bedrooms']


    prediction = model.predict([[size, bedrooms]])[0]


    return jsonify({'price': round(prediction, 2)})

# Function to run Flask in background
def run_flask():
    app.run(host='127.0.0.1', port=5000, debug=False, use_reloader=False)

# Start Flask in a background thread
thread = threading.Thread(target=run_flask)
thread.daemon = True
thread.start()

# Wait a moment for server to start
time.sleep(2)
print("✅ Flask API running on http://127.0.0.1:5000")
```

```
 * Serving Flask app '__main__'
 * Debug mode: off
INFO:werkzeug:WARNING: This is a development server. Do not use it in a pr
oduction deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
INFO:werkzeug:Press CTRL+C to quit
✅ Flask API running on http://127.0.0.1:5000
```

Step 3 : Testing Flask API using Requests Library

We use the **Python** `requests` **library** to send a POST request to the Flask API and retrieve the predicted house price.

Import Library

```
import requests
```

- `requests` is used to send HTTP requests to APIs

API Request

We send a POST request to the prediction endpoint:

$$\text{URL} = \text{http://localhost:5000/predict}$$

$$\text{Method} = \text{POST}$$

```
response = requests.post(
    'http://localhost:5000/predict',
    json={'size': 2100, 'bedrooms': 5}
)
```

- Sends input data in JSON format
- Features:
  - $x_1 = 2100$ (size)
  - $x_2 = 5$ (bedrooms)

Input Representation

$$X = \begin{bmatrix} 2100 & 5 \end{bmatrix}$$

Model Prediction

The API internally computes:

$$\hat{y} = f(2100, 5)$$

Extract Response

`response.json()`

- Converts API response into Python dictionary

Display Output

```
print(f"Predicted price: ${response.json()['price']:,.2f}")
```

- Extracts predicted price
- Formats output with commas and 2 decimal places

Output Example

$$\text{Predicted Price} = 420000.00$$

Console output:

```
Predicted price: $420,000.00
```

In [ ]:

```
import requests

# Test prediction
response = requests.post(
    'http://localhost:5000/predict',
    json={'size': 2100, 'bedrooms': 5}
)

print(f"Predicted price: ${response.json()['price']:,.2f}")
```

Conclusion

We've successfully walked through the entire journey of creating, deploying, and testing a straightforward **Linear Regression model** to predict house prices in this notebook. It's been a great way to see an end-to-end machine learning project in action!

Here's a quick recap of what we covered:

1. **Training the Model**: We started by training a `LinearRegression` model from `sklearn` using a small dataset of house sizes and bedrooms to learn how to predict prices.
2. **Saving Our Work**: To avoid retraining every time, we saved our trained model efficiently using `joblib`.
3. **Building an API**: Then, we set up a **Flask API**, running it seamlessly in a background thread, so our saved model could start making predictions on demand.
4. **Checking it Works**: Finally, we tested our API using the `requests` library, sending some sample house data and getting back accurate price predictions.

This whole process gives us a clear, functional example of how a machine learning model can be brought to life as a web service for real-time predictions. It really highlights the power of model saving, integrating with web frameworks, and interacting with APIs to create practical solutions.